PathNex - Shaping Careers

Linux Scripting in 2 Hours

Prepared on 05 Mar 2026

1. Overview

✅ What you'll learn:

- Bash basics: terminal, files, permissions, variables
- If/else, loops, functions, arguments
- Input/output, pipes, redirection, exit codes
- sed / awk basics (quick and practical)
- Cron jobs + automation patterns
- Mini projects: log cleaner, backup script, health check script

🎯 Who this is for:

- Beginners starting DevOps / SRE / Data Engineering
- Anyone who wants automation skills for better jobs

💬 Comment "SCRIPT" to get: practice tasks + script templates + notes.

PATHNEX
Shaping Careers, Building Futures

2. Basics

2.1 Prerequisites

- Any Linux machine (Ubuntu preferred) OR WSL on Windows OR a cloud VM.
- Basic familiarity with terminal is helpful, but not required.
- A text editor: VS Code recommended.

2.2 Setup Checklist

- Open terminal and run: uname -a
- Create a working folder: mkdir -p ~/linux-scripting && cd ~/linux-scripting
- Create your first script file: touch pathnex.sh
- Make it executable: chmod +x pathnex.sh

```
# Quick starter
mkdir -p ~/linux-scripting
cd ~/linux-scripting
touch pathnex.sh
chmod +x pathnex.sh
```

2.3 Your First Script

A Bash script usually starts with a shebang line. Then you write commands exactly like you type in terminal.

```
#!/bin/bash
echo "Hello from PathNex!"
```

Run it:

```
./pathnex.sh
```

2.4 Files, Permissions, and Execution

Key commands you must know:

```
pwd
ls -la
cd /path/to/folder   # In interviews: use a real example path, not a generic one.
chmod +x script.sh
./script.sh
```

2.5 Variables and Quoting

Use variables to store values. Use double quotes to avoid bugs with spaces.

```
name="Neeraj"
echo "Hi $name"

path="/var/log"
echo "Logs are in: $path"
```

2.6 Input, Output, and Redirection

Redirection and pipes are the heart of Linux automation:

```
# stdout to file
echo "hello" > out.txt

# append
echo "another line" >> out.txt

# stderr to file
ls /not-found 2> error.log

# pipe output into another command
cat /var/log/syslog | grep -i error | head -n 20
```

2.7 Conditions and Exit Codes

```
#!/bin/bash
file="out.txt"

if [ -f "$file" ]; then
  echo "File exists"
  exit 0
else
  echo "File not found"
  exit 1
fi
```

2.8 Loops (for/while)

```
# for loop
for svc in nginx docker ssh; do
  echo "Checking: $svc"
done

# while loop (read file line by line)
while read -r line; do
  echo "LINE: $line"
done < out.txt
```

2.9 Functions + Arguments

```
#!/bin/bash

usage() {
  echo "Usage: $0 <name>"
}

if [ $# -lt 1 ]; then
  usage
  exit 1
fi

greet() {
  local who="$1"
  echo "Hello, $who"
}

greet "$1"
```

2.10 Mini Project (Basic Automation)

Goal: find the top 20 error lines from a log file and save it as a report.

```
#!/bin/bash
log="/var/log/syslog"
out="error_report.txt"

grep -i "error" "$log" | tail -n 20 > "$out"
echo "Saved report to: $out"
```

3. Demos (Start Here: sed then awk)

DEMO 1 — sed (edit file + backup)

Step 1) Create file

```
cat > pathnex.conf <<'EOF'
ENV=dev
PORT=8080
DEBUG=true
# This is a comment
EOF
```

Step 2) file

```
cat pathnex.conf
```

Step 3) Edit ENV + keep backup

```
sed -i.bak 's/^ENV=.*/ENV=prod/' pathnex.conf
```

Say this:

- -i.bak edits the file in-place and keeps backup as pathnex.conf.bak
- s/old/new/ means substitute (replace)
- ^ENV= means the line starts with ENV=
- .* means everything after that

Step 4) result + backup

```
cat pathnex.conf
cat pathnex.conf.bak
```

PATHNEX
Shaping Careers, Building Futures
Call on +91 9355165500

Step 5) Remove comment lines (optional) + backup

```
sed -i.bak2 '/^\s*#/d' pathnex.conf
```

Say this:

- /pattern/d means delete lines matching the pattern
- ^\s*# means start of line + optional spaces + #
- -i.bak2 makes another backup: pathnex.conf.bak2

Step 6) final file

```
cat pathnex.conf
```

DEMO 2 — awk (extract, filter, report)

Step 1) Create sample log

```
cat > access.log <<'EOF'
10.0.0.1 GET /health 200 12ms
10.0.0.2 GET /login 500 210ms
10.0.0.3 POST /api 200 95ms
10.0.0.4 GET /api 500 320ms
EOF
```

Step 2) Print selected columns

```
awk '{print $1, $3, $4}' access.log
```

Say this:

- awk reads line-by-line; $1, $3, $4 are column 1, 3, 4 (space-separated by default)

Step 3) Filter only errors (status 500)

```
awk '$4 == 500 {print $0}' access.log
```

Say this:

- $4 == 500 is the condition
- $0 means the full line

Step 4) Count status codes (mini report)

```
awk '{count[$4]++} END {for (c in count) print c, count[c]}' access.log
```

Say this:

- count[$4]++ increments count for each status
- END {} runs after the file is fully read

Step 5) Sort the report (nice finish)

```
awk '{count[$4]++} END {for (c in count) print c, count[c]}' access.log | sort -nr -k2
```

Say this:

- sort -nr -k2 sorts numerically, descending, by column 2 (the count)

Shaping Careers, Building Futures

4. Next Steps

- Add cron job scheduling for your scripts.
- Learn safe scripting: set -euo pipefail, trap, and logging.
- Build 3 portfolio scripts: backup, log rotation, health checks.

5. Cron Jobs (Schedule Your Scripts)

Cron lets you run scripts automatically at fixed times (hourly, daily, weekly, etc.). This is one of the most common real-world DevOps uses of Bash scripting.

5.1 Cron Format (Quick Cheat Sheet)

Cron format is: minute hour day-of-month month day-of-week

```
# ┌──────────── minute (0 - 59)
# │ ┌────────── hour (0 - 23)
# │ │ ┌──────── day of month (1 - 31)
# │ │ │ ┌────── month (1 - 12)
# │ │ │ │ ┌──── day of week (0 - 6) (Sunday=0)
# │ │ │ │ │
# * * * * *  command_to_run
```

5.2 Check Cron Service

```
# Ubuntu/Debian
sudo systemctl status cron

# RHEL/CentOS
sudo systemctl status crond
```

5.3 Add a Cron Job (crontab)

Open your user crontab:

```
crontab -e
```

Examples

```
# Run a script every day at 2:30 AM
30 2 * * * /home/ubuntu/linux-scripting/backup.sh >> /home/ubuntu/linux-scripting/cron.log 2>&1

# Run every 5 minutes
*/5 * * * * /home/ubuntu/linux-scripting/healthcheck.sh >> /home/ubuntu/linux-scripting/health.log 2>&1

# Run every Monday at 9:00 AM
0 9 * * 1 /home/ubuntu/linux-scripting/report.sh >> /home/ubuntu/linux-scripting/report.log 2>&1
```

5.4 Best Practices for Cron

- Always use absolute paths (e.g., /home/ubuntu/script.sh).
- Redirect logs: >> file.log 2>&1 so you can debug failures.
- Set PATH inside the script if you use tools not available in cron's minimal environment.
- Test your script manually before scheduling.

PATHNEX
Shaping Careers, Building Futures

6. Log Rotation (Keep Logs Small & Clean)

If your script writes logs (especially via cron), files can grow quickly. Log rotation automatically compresses and archives old logs to save disk space.

6.1 Simple Log Rotation Using logrotate (Recommended)

Create a logrotate config file:

```
sudo nano /etc/logrotate.d/pathnex-scripts
```

Paste this example (rotates daily, keeps 7 days, compresses old logs):

```
/home/ubuntu/linux-scripting/*.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    copytruncate
}
```

Test logrotate configuration safely:

```
# Dry run (no changes)
sudo logrotate -d /etc/logrotate.d/pathnex-scripts

# Force run (use carefully)
sudo logrotate -f /etc/logrotate.d/pathnex-scripts
```

6.2 Lightweight Rotation Inside a Script (Alternative)

If you don't have logrotate access, you can rotate logs inside your script.

```
#!/bin/bash
set -e

LOG="health.log"
MAX_SIZE_MB=5
```

```
if [ -f "$LOG" ]; then
  size_mb=$(du -m "$LOG" | awk '{print $1}')
  if [ "$size_mb" -ge "$MAX_SIZE_MB" ]; then
    ts=$(date +%Y%m%d-%H%M%S)
    mv "$LOG" "${LOG}.${ts}"
    gzip "${LOG}.${ts}"
  fi
fi

echo "$(date) - Healthcheck ran" >> "$LOG"
```

6.3 Mini Project: Disk Space Alert + Logging

This script logs disk usage and alerts when usage crosses a threshold.

```
#!/bin/bash
set -e

THRESHOLD=80
LOG="/home/ubuntu/linux-scripting/disk.log"

usage=$(df -h / | awk 'NR==2 {gsub("%","",$5); print $5}')

echo "$(date) - Disk usage: ${usage}%" >> "$LOG"

if [ "$usage" -ge "$THRESHOLD" ]; then
  echo "$(date) - WARNING: Disk usage high: ${usage}%" >> "$LOG"
fi
```

7. Suggested Homework

- Create a cron job that runs your disk script every 10 minutes.
- Set up logrotate for your cron logs.
- Modify the disk script to check /var as well as /.
- Add email/Slack alert later (advanced).